

Contexto Técnico: Prevención de SQL Injection en APIs REST

Definición y Riesgo

La inyección SQL (SQL Injection) es una vulnerabilidad de seguridad crítica que permite a atacantes manipular consultas SQL mediante la inserción de código malicioso a través de parámetros de entrada no validados. En APIs REST, esta vulnerabilidad puede comprometer la confidencialidad, integridad y disponibilidad de los datos.

Vectores de Ataque Comunes en APIs REST

1. Parámetros de Query String

GET /api/users?id=1' OR '1'=1

GET /api/products?category=electronics' UNION SELECT * FROM users--

2. Cuerpo de Solicitud (Request Body)

```
{  
  "username": "admin' --",  
  "email": "test@example.com' OR '1'=1"  
}
```

3. Encabezados HTTP

X-User-Id: 123' OR '1'=1

Authorization: Bearer token' UNION SELECT password FROM users—

4. Parámetros de Ruta (Path Parameters)

GET /api/orders/1' OR '1'=1/details

DELETE /api/products/'; DROP TABLE products—

Mejores Prácticas de Prevención

1. Consultas Parametrizadas (Prepared Statements)

Incorrecto - Concatenación Directa:

```
// C# ASP.NET Core - VULNERABLE  
  
string query = $"SELECT * FROM Users WHERE Id = {id}";  
  
var result = context.Database.ExecuteSqlRaw(query);
```

Correcto - Consulta Parametrizada:

```
// ADO.NET con SQL Server  
  
string query = "SELECT * FROM Users WHERE Id = @Id";  
using (var command = new SqlCommand(query, connection))  
{  
    command.Parameters.AddWithValue("@Id", userId);  
    using (var reader = command.ExecuteReader())  
    {  
        // Procesar resultados  
    }  
}
```

```
// Entity Framework Core - Raw SQL  
  
string query = "SELECT * FROM Users WHERE Id = {0}";  
  
var users = context.Users.FromSqlRaw(query, userId).ToList();
```

```
// Dapper  
  
string query = "SELECT * FROM Users WHERE Id = @Id";  
  
var user = connection.QueryFirstOrDefault<User>(query, new { Id = userId });
```

2. ORM/Query Builders

Entity Framework Core (Recomendado):

```
// LINQ - Completamente seguro
var user = await context.Users
    .Where(u => u.Id == userId)
    .FirstOrDefaultAsync();

// Find by primary key
var user = await context.Users.FindAsync(userId);

// Con filtros complejos
var users = await context.Users
    .Where(u => u.Email == email && u.IsActive)
    .Include(u => u.Orders)
    .ToListAsync();
```

Dapper (Micro-ORM):

```
// Consulta parametrizada con Dapper
var user = await connection.QueryFirstOrDefaultAsync<User>(
    "SELECT * FROM Users WHERE Id = @Id AND Email = @Email",
    new { Id = userId, Email = email }
);

// Multiple queries
var sql = @"
    SELECT * FROM Users WHERE Id = @Id;
    SELECT * FROM Orders WHERE UserId = @Id;
";
using (var multi = await connection.QueryMultipleAsync(sql, new { Id = userId }))
{
    var user = await multi.ReadFirstOrDefaultAsync<User>();
    var orders = await multi.ReadAsync<Order>(); }
}
```

3. Validación y Sanitización de Entrada

Data Annotations (ASP.NET Core):

```
public class UserDto
```

```

{
    [Required(ErrorMessage = "El ID es requerido")]
    [Range(1, int.MaxValue, ErrorMessage = "El ID debe ser mayor a 0")]
    public int Id { get; set; }

    [Required]
    [EmailAddress(ErrorMessage = "Email inválido")]
    [StringLength(100)]
    public string Email { get; set; }

    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_]{3,30}$",
        ErrorMessage = "Username debe ser alfanumérico y entre 3-30 caracteres")]
    public string Username { get; set; }
}

```

FluentValidation (Recomendado):

```

public class UserValidator : AbstractValidator<UserDto>
{
    public UserValidator()
    {
        RuleFor(x => x.Id)
            .GreaterThan(0)
            .WithMessage("ID debe ser mayor a 0");

        RuleFor(x => x.Email)
            .NotEmpty()
            .EmailAddress()
            .MaximumLength(100);

        RuleFor(x => x.Username)

```

```

        .NotEmpty()
        .Length(3, 30)
        .Matches(@"^[a-zA-Z0-9_]+$")
        .WithMessage("Username solo puede contener letras, números y guiones bajos");
    }
}

```

// Registro en Program.cs

```

builder.Services.AddValidatorsFromAssemblyContaining<UserValidator>();
builder.Services.AddFluentValidationAutoValidation();

```

4. Principio de Privilegio Mínimo

- Usar cuentas de base de datos con permisos restringidos
- Evitar usar usuarios con permisos de administrador para operaciones de aplicación
- Implementar usuarios específicos por módulo/funcionalidad

5. Listas Blancas (Whitelisting)

C# ASP.NET Core:

// Para ordenamiento dinámico

```

private static readonly HashSet<string> AllowedSortFields = new()
{
    "name", "date", "price", "category"
};

```

[HttpGet]

```

public async Task<IActionResult> GetProducts(string sortBy = "name")
{
    // Validar campo de ordenamiento
    if (!AllowedSortFields.Contains(sortBy.ToLower()))
    {
        sortBy = "name"; // valor por defecto seguro
    }
}

```

```
}
```

```
// Usar LINQ (seguro) en lugar de SQL dinámico
```

```
var products = sortBy.ToLower() switch
```

```
{
```

```
    "name" => await context.Products.OrderBy(p => p.Name).ToListAsync(),
```

```
    "date" => await context.Products.OrderBy(p => p.Date).ToListAsync(),
```

```
    "price" => await context.Products.OrderBy(p => p.Price).ToListAsync(),
```

```
    _ => await context.Products.OrderBy(p => p.Name).ToListAsync()
```

```
};
```

```
return Ok(products);
```

```
}
```

```
// Para filtros dinámicos usando Expression Trees
```

```
public IQueryable<Product> ApplyFilter(IQueryable<Product> query, string field, string value)
```

```
{
```

```
    var allowedFields = new Dictionary<string, Expression<Func<Product, bool>>>
```

```
{
```

```
    ["category"] = p => p.Category == value,
```

```
    ["name"] = p => p.Name.Contains(value),
```

```
    ["brand"] = p => p.Brand == value
```

```
};
```

```
if (allowedFields.TryGetValue(field.ToLower(), out var predicate))
```

```
{
```

```
    return query.Where(predicate);
```

```
}
```

```
    return query;
}
```

6. Escape de Caracteres Especiales

```
// Último recurso, preferir consultas parametrizadas
const escapedInput = mysql.escape(userInput);
```

Patrones Específicos para APIs REST en C# / ASP.NET Core

1. Middleware de Validación

Action Filter para validación:

```
// Filtro de validación personalizado
public class ValidateModelAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (!context.ModelState.IsValid)
        {
            var errors = context.ModelState
                .Where(e => e.Value.Errors.Count > 0)
                .Select(e => new
                {
                    Field = e.Key,
                    Errors = e.Value.Errors.Select(x => x.ErrorMessage)
                });

            context.Result = new BadRequestObjectResult(new
            {
                Message = "Errores de validación",
                Errors = errors
            });
        }
    }
}
```

```
}
```

```
// Aplicar globalmente en Program.cs
```

```
builder.Services.AddControllers(options =>
```

```
{
```

```
    options.Filters.Add<ValidateModelAttribute>();
```

```
});
```

```
// O aplicar por controlador/acción
```

```
[ValidateModel]
```

```
[HttpPost]
```

```
public async Task<IActionResult> CreateUser([FromBody] UserDto dto)
```

```
{
```

```
    // El modelo ya está validado aquí
```

```
}
```

Middleware personalizado:

```
public class InputValidationMiddleware
```

```
{
```

```
    private readonly RequestDelegate _next;
```

```
    private readonly ILogger<InputValidationMiddleware> _logger;
```

```
    public InputValidationMiddleware(
```

```
        RequestDelegate next,
```

```
        ILogger<InputValidationMiddleware> logger)
```

```
    {
```

```
        _next = next;
```

```
        _logger = logger;
```

```
    }
```

```
    public async Task InvokeAsync(HttpContext context)
```



```

{
    // Validar route parameters
    foreach (var param in context.Request.RouteValues)
    {
        if (param.Key.EndsWith("Id", StringComparison.OrdinalIgnoreCase))
        {
            if (!int.TryParse(param.Value?.ToString(), out var id) || id <= 0)
            {
                _logger.LogWarning(
                    "Invalid ID parameter: {Key}={Value}",
                    param.Key,
                    param.Value);

                context.Response.StatusCode = 400;
                await context.Response.WriteAsJsonAsync(new
                {
                    error = $"Parámetro {param.Key} inválido"
                });
                return;
            }
        }
    }

    await _next(context);
}
}

```

```

// Registro en Program.cs
app.UseMiddleware<InputValidationMiddleware>();

```

2. Validación con Model Binding

Controlador con validación automática:

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class UsersController : ControllerBase
```

```
{
```

```
    private readonly AppDbContext _context;
```

```
    private readonly ILogger<UsersController> _logger;
```

```
    public UsersController(AppDbContext context, ILogger<UsersController> logger)
```

```
    {
```

```
        _context = context;
```

```
        _logger = logger;
```

```
    }
```

```
// Validación automática de parámetros de ruta
```

```
[HttpGet("{id:int:min(1)}")]
```

```
public async Task<ActionResult<UserDto>> GetUser(int id)
```

```
{
```

```
    var user = await _context.Users
```

```
        .Where(u => u.Id == id)
```

```
        .Select(u => new UserDto
```

```
        {
```

```
            Id = u.Id,
```

```
            Email = u.Email,
```

```
            Username = u.Username
```

```
        })
```

```
        .FirstOrDefaultAsync();
```

```
    if (user == null)
```

```
        return NotFound();
```

```

        return Ok(user);
    }

    // Validación de body con Data Annotations
    [HttpPost]
    public async Task<ActionResult<UserDto>> CreateUser(
        [FromBody] CreateUserDto dto)
    {
        // ModelState se valida automáticamente con [ApiController]

        var user = new User
        {
            Email = dto.Email,
            Username = dto.Username,
            CreatedAt = DateTime.UtcNow
        };

        _context.Users.Add(user);
        await _context.SaveChangesAsync();

        return CreatedAtAction(
            nameof(GetUser),
            new { id = user.Id },
            user);
    }

    // Validación de query parameters
    [HttpGet]
    public async Task<ActionResult<PagedResult<UserDto>>> GetUsers(

```

```

[FromQuery] UserQueryParameters parameters)
{
    var query = _context.Users.AsQueryable();

    // Aplicar filtros de manera segura
    if (!string.IsNullOrEmpty(parameters.Search))
    {
        query = query.Where(u =>
            u.Username.Contains(parameters.Search) ||
            u.Email.Contains(parameters.Search));
    }

    // Ordenamiento seguro con whitelist
    query = parameters.SortBy?.ToLower() switch
    {
        "email" => parameters.SortDescending
            ? query.OrderByDescending(u => u.Email)
            : query.OrderBy(u => u.Email),
        "createdat" => parameters.SortDescending
            ? query.OrderByDescending(u => u.CreatedAt)
            : query.OrderBy(u => u.CreatedAt),
        _ => query.OrderBy(u => u.Username)
    };

    // Paginación
    var total = await query.CountAsync();
    var items = await query
        .Skip((parameters.Page - 1) * parameters.PageSize)
        .Take(parameters.PageSize)
        .Select(u => new UserDto

```

```

        {
            Id = u.Id,
            Email = u.Email,
            Username = u.Username
        })
        .ToListAsync();

return Ok(new PagedResult<UserDto>
{
    Items = items,
    TotalCount = total,
    Page = parameters.Page,
    PageSize = parameters.PageSize
});
}
}

```

// DTOs con validación

```

public class CreateUserDto
{
    [Required]
    [EmailAddress]
    [MaxLength(100)]
    public string Email { get; set; }

    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_]{3,30}$")]
    public string Username { get; set; }

    [Required]

```

```

[MinLength(8)]

[RegularExpression(@"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]",

    ErrorMessage = "Password debe contener mayúscula, minúscula, número y carácter especial")]

    public string Password { get; set; }
}

public class UserQueryParameters
{
    [Range(1, int.MaxValue)]
    public int Page { get; set; } = 1;

    [Range(1, 100)]
    public int PageSize { get; set; } = 10;

    [MaxLength(50)]
    public string? Search { get; set; }

    [RegularExpression(@"^(username|email|createdat)$",
        ErrorMessage = "SortBy debe ser: username, email o createdat")]
    public string? SortBy { get; set; }

    public bool SortDescending { get; set; }
}

```

3. Rate Limiting y Detección de Anomalías

ASP.NET Core Rate Limiting (Native - .NET 7+):

```

// Program.cs

using System.Threading.RateLimiting;

builder.Services.AddRateLimiter(options =>

```

```

{
    // Fixed Window Limiter
    options.AddFixedWindowLimiter("fixed", options =>
    {
        options.PermitLimit = 100;
        options.Window = TimeSpan.FromMinutes(15);
        options.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        options.QueueLimit = 5;
    });

    // Sliding Window Limiter (más preciso)
    options.AddSlidingWindowLimiter("sliding", options =>
    {
        options.PermitLimit = 100;
        options.Window = TimeSpan.FromMinutes(15);
        options.SegmentsPerWindow = 3;
    });

    // Token Bucket (para burst traffic)
    options.AddTokenBucketLimiter("token", options =>
    {
        options.TokenLimit = 100;
        options.ReplenishmentPeriod = TimeSpan.FromMinutes(1);
        options.TokensPerPeriod = 20;
    });

    // Concurrency Limiter
    options.AddConcurrencyLimiter("concurrent", options =>
    {
        options.PermitLimit = 10;
    });
}

```

```

options.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
options.QueueLimit = 5;
});

// Política por IP
options.AddPolicy("perIp", context =>
{
    var ipAddress = context.Connection.RemoteIpAddress?.ToString() ?? "unknown";

    return RateLimitPartition.GetFixedWindowLimiter(ipAddress, _ =>
        new FixedWindowRateLimiterOptions
        {
            PermitLimit = 50,
            Window = TimeSpan.FromMinutes(1)
        });
});

options.OnRejected = async (context, token) =>
{
    context.HttpContext.Response.StatusCode = 429;
    await context.HttpContext.Response.WriteAsync(
        "Demasiadas solicitudes. Intente nuevamente más tarde.",
        cancellationToken: token);
};
});

app.UseRateLimiter();

// Aplicar en controladores
[EnableRateLimiting("fixed")]

```



```

[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    // Todos los endpoints usan el límite "fixed"
}

// O por endpoint específico
[EnableRateLimiting("perIp")]
[HttpPost]
public async Task<IActionResult> CreateUser([FromBody] CreateUserDto dto)
{
    // ...
}

```

```

// Deshabilitar rate limiting en endpoint específico
[DisableRateLimiting]
[HttpGet("health")]
public IActionResult Health() => Ok();

```

AspNetCoreRateLimit (Librería con más features):

```

// Install: dotnet add package AspNetCoreRateLimit

// Program.cs
builder.Services.AddMemoryCache();
builder.Services.Configure<IpRateLimitOptions>(options =>
{
    options.EnableEndpointRateLimiting = true;
    options.StackBlockedRequests = false;
    options.HttpStatusCode = 429;
    options.RealIpHeader = "X-Real-IP";
}

```

```
options.ClientIdHeader = "X-ClientId";
```

```
options.GeneralRules = new List<RateLimitRule>
{
    new RateLimitRule
    {
        Endpoint = "*",
        Period = "15m",
        Limit = 100
    },
    new RateLimitRule
    {
        Endpoint = "POST:/api/*",
        Period = "1m",
        Limit = 10
    },
    new RateLimitRule
    {
        Endpoint = "GET:/api/users",
        Period = "1s",
        Limit = 5
    }
};
});
```

```
builder.Services.AddInMemoryRateLimiting();
```

```
builder.Services.AddSingleton<IRateLimitConfiguration, RateLimitConfiguration>();
```

```
app.UseIpRateLimiting();
```

Detección de Patrones Sospechosos:

```

public class SqlInjectionDetectionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<SqlInjectionDetectionMiddleware> _logger;

    private static readonly string[] SqlInjectionPatterns = new[]
    {
        @"(\b(SELECT|INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|EXEC|EXECUTE)\b)",
        @"(--|#|\\*|\\*V)",
        @"(\bOR\b.*=.*)",
        @"(\bAND\b.*=.*)",
        @"('|;|--|\\|&)",
        @"(\bUNION\b.*\bSELECT\b)",
        @"(\bEXEC\b.*\()",
        @"(xp_cmdshell|sp_executesql)"
    };

    public SqlInjectionDetectionMiddleware(
        RequestDelegate next,
        ILogger<SqlInjectionDetectionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        // Verificar query string
        if (ContainsSqlInjectionPattern(context.Request.QueryString.Value))

```

```

{
    _logger.LogWarning(
        "Posible intento de SQL Injection detectado en query string. IP: {IP}, Path: {Path}, Query: {Query}",
        context.Connection.RemoteIpAddress,
        context.Request.Path,
        context.Request.QueryString);

    context.Response.StatusCode = 400;
    await context.Response.WriteAsJsonAsync(new
    {
        error = "Solicitud inválida"
    });
    return;
}

// Verificar route values
foreach (var value in context.Request.RouteValues.Values)
{
    if (value != null && ContainsSqlInjectionPattern(value.ToString()))
    {
        _logger.LogWarning(
            "Posible intento de SQL Injection detectado en parámetros de ruta. IP: {IP}",
            context.Connection.RemoteIpAddress);

        context.Response.StatusCode = 400;
        await context.Response.WriteAsJsonAsync(new
        {
            error = "Parámetros inválidos"
        });
    }
}

```

```

        return;
    }
}

await _next(context);
}

private bool ContainsSqlInjectionPattern(string? input)
{
    if (string.IsNullOrEmpty(input))
        return false;

    return SqlInjectionPatterns.Any(pattern =>
        Regex.IsMatch(input, pattern, RegexOptions.IgnoreCase));
}
}

// Registro
app.UseMiddleware<SqlInjectionDetectionMiddleware>();

```

Capas de Defensa (Defense in Depth)

1. **Capa de Aplicación:** Validación de entrada, consultas parametrizadas
2. **Capa de Base de Datos:** Privilegios mínimos, stored procedures seguros
3. **Capa de Red:** WAF (Web Application Firewall), IDS/IPS
4. **Capa de Monitoreo:** Logging de consultas sospechosas, alertas

Herramientas de Detección y Prevención para .NET

Análisis Estático de Código

- **SonarQube / SonarCloud:** Análisis de vulnerabilidades en C#
- **Checkmarx:** SAST específico para .NET
- **Snyk Code:** Detección de vulnerabilidades en tiempo real
- **Security Code Scan:** Analizador Roslyn para seguridad en .NET

- **Puma Scan:** Analizador de seguridad para Visual Studio

Instalación Security Code Scan:

dotnet add package SecurityCodeScan.VS2019

Pruebas de Penetración

- **OWASP ZAP:** Testing de seguridad para APIs
- **Burp Suite:** Análisis de vulnerabilidades web
- **SQLMap:** Específico para SQL Injection
- **Postman:** Testing manual de APIs

Monitoreo en Tiempo Real

- **Azure Application Insights:** Monitoreo y alertas
- **Serilog con enrichers de seguridad:** Logging estructurado
- **Seq:** Análisis de logs en tiempo real
- **ELK Stack:** Elasticsearch, Logstash, Kibana

Checklist de Seguridad para APIs REST en C# / ASP.NET Core

- ☐ Usar Entity Framework Core, Dapper o consultas parametrizadas (NUNCA concatenación)
- ☐ Validar todos los inputs usando FluentValidation o Data Annotations
- ☐ Aplicar constraints en route parameters ({id:int:min(1)})
- ☐ Implementar whitelist para ordenamiento y filtros dinámicos
- ☐ Configurar DbContext con privilegios mínimos en SQL Server
- ☐ Habilitar [ApiController] para validación automática de ModelState
- ☐ Implementar rate limiting (.NET 7+ nativo o AspNetCoreRateLimit)
- ☐ Agregar middleware de detección de patrones SQL injection
- ☐ Usar logging estructurado (Serilog, NLog) para auditoría
- ☐ Implementar respuestas genéricas de error (no revelar estructura de BD)
- ☐ Configurar CORS restrictivo
- ☐ Habilitar HTTPS obligatorio
- ☐ Implementar autenticación JWT o OAuth2
- ☐ Usar secretos seguros (Azure Key Vault, AWS Secrets Manager)
- ☐ Realizar code review enfocado en seguridad

- [] Ejecutar análisis estático con Security Code Scan o SonarQube
- [] Realizar pruebas de penetración con OWASP ZAP
- [] Mantener dependencias NuGet actualizadas
- [] Configurar health checks y monitoring
- [] Documentar API con Swagger/OpenAPI incluyendo validaciones

Ejemplo de Implementación Completa en C# / ASP.NET Core

```
// ===== Program.cs =====

using FluentValidation;
using Microsoft.EntityFrameworkCore;
using System.Threading.RateLimiting;

var builder = WebApplication.CreateBuilder(args);

// Configurar DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions => sqlOptions.EnableRetryOnFailure()));

// Validación con FluentValidation
builder.Services.AddValidatorsFromAssemblyContaining<Program>();
builder.Services.AddFluentValidationAutoValidation();

// Rate Limiting
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("api", opt =>
    {
        opt.PermitLimit = 100;
        opt.Window = TimeSpan.FromMinutes(1);
    });
});
```

```

    });
});

// Configurar controladores con validación automática
builder.Services.AddControllers(options =>
{
    options.Filters.Add<ValidateModelAttribute>();
});

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseRateLimiter();
app.UseMiddleware<SqlInjectionDetectionMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();

// ===== Models/User.cs =====

```



```

public class User
{
    public int Id { get; set; }

    [MaxLength(100)]
    public string Email { get; set; } = string.Empty;

    [MaxLength(50)]
    public string Username { get; set; } = string.Empty;

    public DateTime CreatedAt { get; set; }
    public bool IsActive { get; set; } = true;
}

// ===== DTOs/UserDtos.cs =====

public class CreateUserDto
{
    public string Email { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class UpdateUserDto
{
    public string? Email { get; set; }
    public string? Username { get; set; }
}

public class UserResponseDto
{

```

```

public int Id { get; set; }

public string Email { get; set; } = string.Empty;

public string Username { get; set; } = string.Empty;

public DateTime CreatedAt { get; set; }
}

// ===== Validators/UserValidators.cs =====

public class CreateUserDtoValidator : AbstractValidator<CreateUserDto>
{
    public CreateUserDtoValidator()
    {
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email es requerido")
            .EmailAddress().WithMessage("Email inválido")
            .MaxLength(100).WithMessage("Email demasiado largo");

        RuleFor(x => x.Username)
            .NotEmpty().WithMessage("Username es requerido")
            .Length(3, 30).WithMessage("Username debe tener entre 3 y 30 caracteres")
            .Matches(@"^[a-zA-Z0-9_]+$")
            .WithMessage("Username solo puede contener letras, números y guiones
bajos");

        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Password es requerido")
            .MinimumLength(8).WithMessage("Password debe tener al menos 8 caracteres")
            .Matches(@"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]")
            .WithMessage("Password debe contener mayúscula, minúscula, número y
carácter especial");
    }
}

```

```

public class UpdateUserDtoValidator : AbstractValidator<UpdateUserDto>
{
    public UpdateUserDtoValidator()
    {
        When(x => !string.IsNullOrEmpty(x.Email), () =>
        {
            RuleFor(x => x.Email)
                .EmailAddress().WithMessage("Email inválido")
                .MaxLength(100);
        });

        When(x => !string.IsNullOrEmpty(x.Username), () =>
        {
            RuleFor(x => x.Username)
                .Length(3, 30)
                .Matches(@"^[a-zA-Z0-9_]+$");
        });
    }
}

```

// ===== Data/AppDbContext.cs =====

```

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<User> Users { get; set; }
}

```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<User>(entity =>
    {
        entity.HasKey(e => e.Id);
        entity.HasIndex(e => e.Email).IsUnique();
        entity.HasIndex(e => e.Username).IsUnique();

        entity.Property(e => e.Email).IsRequired();
        entity.Property(e => e.Username).IsRequired();
    });
}
}

```

```

// ===== Controllers/UsersController.cs =====

```

```

[EnableRateLimiting("api")]
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly ILogger<UsersController> _logger;

    public UsersController(
        AppDbContext context,
        ILogger<UsersController> logger)
    {

```

```
_context = context;
_logger = logger;
}
```

```
/// <summary>
```

```
/// Obtiene un usuario por ID
```

```
/// </summary>
```

```
[HttpGet("{id:int:min(1)}")]
```

```
[ProducesResponseType(typeof(UserResponseDto), StatusCodes.Status200OK)]
```

```
[ProducesResponseType(StatusCodes.Status404NotFound)]
```

```
public async Task<ActionResult<UserResponseDto>> GetUser(int id)
```

```
{
```

```
    try
```

```
    {
```

```
        // Uso seguro de EF Core - NO hay concatenación de strings
```

```
        var user = await _context.Users
```

```
            .Where(u => u.Id == id && u.IsActive)
```

```
            .Select(u => new UserResponseDto
```

```
            {
```

```
                Id = u.Id,
```

```
                Email = u.Email,
```

```
                Username = u.Username,
```

```
                CreatedAt = u.CreatedAt
```

```
            })
```

```
            .FirstOrDefaultAsync();
```

```
        if (user == null)
```

```
        {
```

```
            _logger.LogInformation("Usuario {UserId} no encontrado", id);
```

```
            return NotFound(new { message = "Usuario no encontrado" });
```

```

    }

    return Ok(user);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error al obtener usuario {UserId}", id);
    return StatusCode(500, new { message = "Error interno del servidor" });
}
}

/// <summary>
/// Crea un nuevo usuario
/// </summary>
[HttpPost]
[ProducesResponseType(typeof(UserResponseDto), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<ActionResult<UserResponseDto>> CreateUser(
    [FromBody] CreateUserDto dto)
{
    try
    {
        // Verificar si el email o username ya existe
        var exists = await _context.Users
            .AnyAsync(u => u.Email == dto.Email || u.Username == dto.Username);

        if (exists)
        {
            return BadRequest(new
            {

```

```

        message = "Email o username ya está en uso"
    });
}

// Crear usuario - usando parámetros, no concatenación
var user = new User
{
    Email = dto.Email,
    Username = dto.Username,
    CreatedAt = DateTime.UtcNow,
    IsActive = true
    // Password debe ser hashado con BCrypt, Argon2, etc.
};

_context.Users.Add(user);
await _context.SaveChangesAsync();

var response = new UserResponseDto
{
    Id = user.Id,
    Email = user.Email,
    Username = user.Username,
    CreatedAt = user.CreatedAt
};

_logger.LogInformation(
    "Usuario creado exitosamente: {UserId}",
    user.Id);

return CreatedAtAction(

```

```

        nameof(GetUser),
        new { id = user.Id },
        response);
    }
    catch (DbUpdateException ex)
    {
        _logger.LogError(ex, "Error de base de datos al crear usuario");
        return StatusCode(500, new
        {
            message = "Error al crear usuario"
        });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error inesperado al crear usuario");
        return StatusCode(500, new
        {
            message = "Error interno del servidor"
        });
    }
}

```

/// <summary>

/// Actualiza un usuario existente

/// </summary>

[HttpPut("{id:int:min(1)}")]

[ProducesResponseType(typeof(UserResponseDto), StatusCodes.Status200OK)]

[ProducesResponseType(StatusCodes.Status404NotFound)]

public async Task<ActionResult<UserResponseDto>> UpdateUser(

int id,


```

[FromBody] UpdateUserDto dto)
{
    try
    {
        var user = await _context.Users
            .FirstOrDefaultAsync(u => u.Id == id && u.IsActive);

        if (user == null)
            return NotFound(new { message = "Usuario no encontrado" });

        // Actualización segura con EF Core
        if (!string.IsNullOrEmpty(dto.Email))
            user.Email = dto.Email;

        if (!string.IsNullOrEmpty(dto.Username))
            user.Username = dto.Username;

        await _context.SaveChangesAsync();

        var response = new UserResponseDto
        {
            Id = user.Id,
            Email = user.Email,
            Username = user.Username,
            CreatedAt = user.CreatedAt
        };

        return Ok(response);
    }
    catch (Exception ex)

```

```

{
    _logger.LogError(ex, "Error al actualizar usuario {UserId}", id);
    return StatusCode(500, new
    {
        message = "Error interno del servidor"
    });
}
}

```

```

/// <summary>
/// Busca usuarios con filtros seguros
/// </summary>
[HttpGet]
[ProducesResponseType(typeof(List<UserResponseDto>), StatusCodes.Status200OK)]
public async Task<ActionResult<List<UserResponseDto>>> SearchUsers(
    [FromQuery] string? search,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 10)
{
    try
    {
        // Validar parámetros de paginación
        if (page < 1) page = 1;
        if (pageSize < 1 || pageSize > 100) pageSize = 10;

        var query = _context.Users
            .Where(u => u.IsActive)
            .AsQueryable();

        // Búsqueda segura usando LINQ
    }
}

```

```

if (!string.IsNullOrEmpty(search))
{
    // EF Core parametriza automáticamente
    query = query.Where(u =>
        u.Username.Contains(search) ||
        u.Email.Contains(search));
}

var users = await query
    .OrderBy(u => u.Username)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .Select(u => new UserResponseDto
    {
        Id = u.Id,
        Email = u.Email,
        Username = u.Username,
        CreatedAt = u.CreatedAt
    })
    .ToListAsync();

return Ok(users);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error al buscar usuarios");
    return StatusCode(500, new
    {
        message = "Error interno del servidor"
    });
}

```

```

    }
}

/// <summary>
/// Elimina un usuario (soft delete)
/// </summary>
[HttpDelete("{id:int:min(1)}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<ActionResult> DeleteUser(int id)
{
    try
    {
        // Uso de ExecuteUpdateAsync (EF Core 7+) - seguro
        var rowsAffected = await _context.Users
            .Where(u => u.Id == id && u.IsActive)
            .ExecuteUpdateAsync(setters =>
                setters.SetProperty(u => u.IsActive, false));

        if (rowsAffected == 0)
            return NotFound(new { message = "Usuario no encontrado" });

        _logger.LogInformation("Usuario {UserId} eliminado", id);
        return NoContent();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error al eliminar usuario {UserId}", id);
        return StatusCode(500, new
        {

```

```

        message = "Error interno del servidor"
    });
}
}
}

// ===== Filters/ValidateModelAttribute.cs =====
public class ValidateModelAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (!context.ModelState.IsValid)
        {
            var errors = context.ModelState
                .Where(e => e.Value?.Errors.Count > 0)
                .ToDictionary(
                    e => e.Key,
                    e => e.Value!.Errors.Select(x => x.ErrorMessage).ToArray()
                );

            context.Result = new BadRequestObjectResult(new
            {
                message = "Errores de validación",
                errors
            });
        }
    }
}

```

Configuración de Seguridad en appsettings.json y Program.cs

// appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=MyDb;User
Id=api_user;Password=***;TrustServerCertificate=true;Application Name=MyAPI"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning"
    }
  },
  "RateLimiting": {
    "PermitLimit": 100,
    "Window": "00:01:00"
  },
  "Security": {
    "EnableSqlInjectionDetection": true,
    "LogSuspiciousActivity": true,
    "MaxRequestBodySize": 5242880
  }
}

// Program.cs - Configuración completa de seguridad
var builder = WebApplication.CreateBuilder(args);

// Configurar tamaño máximo de request
builder.Services.Configure<FormOptions>(options =>
{
  options.MultipartBodyLengthLimit = 5242880; // 5MB
});

```

```
builder.WebHost.ConfigureKestrel(options =>
{
    options.Limits.MaxRequestBodySize = 5242880;
});
```

// DbContext con retry policy y timeout

```
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions =>
        {
            sqlOptions.EnableRetryOnFailure(
                maxRetryCount: 3,
                maxRetryDelay: TimeSpan.FromSeconds(5),
                errorNumbersToAdd: null);
            sqlOptions.CommandTimeout(30);
        });
});
```

// Solo en desarrollo

```
if (builder.Environment.IsDevelopment())
{
    options.EnableSensitiveDataLogging();
    options.EnableDetailedErrors();
}
});
```

// CORS restrictivo

```
builder.Services.AddCors(options =>
```

```
{
    options.AddPolicy("ApiPolicy", policy =>
    {
        policy.WithOrigins("https://myapp.com")
            .WithMethods("GET", "POST", "PUT", "DELETE")
            .WithHeaders("Content-Type", "Authorization")
            .AllowCredentials();
    });
});
```

// Configurar controladores con opciones de seguridad

```
builder.Services.AddControllers(options =>
{
    options.Filters.Add<ValidateModelAttribute>();
    options.MaxModelBindingCollectionSize = 100;
})
.ConfigureApiBehaviorOptions(options =>
{
    options.SuppressMapClientErrors = true;
});
```

```
var app = builder.Build();
```

// Configurar pipeline de seguridad

```
if (!app.Environment.IsDevelopment())
{
    app.UseHsts();
}
```

```
app.UseHttpsRedirection();
```



```
app.UseCors("ApiPolicy");  
app.UseRateLimiter();  
app.UseMiddleware<SqlInjectionDetectionMiddleware>();  
app.UseAuthentication();  
app.UseAuthorization();
```

```
app.MapControllers();  
app.Run();
```

- **OWASP Top 10:** A03:2021 – Injection
- **CWE-89:** SQL Injection
- **NIST SP 800-53:** Control de Inyección de Código
- **PCI DSS:** Requisito 6.5.1 - Prevención de Inyección

Consideraciones Adicionales para C# / ASP.NET Core

- **Implementar Content Security Policy (CSP)** en headers HTTP
- **Usar HTTPS obligatorio** con HSTS habilitado
- **Implementar autenticación robusta:** JWT, OAuth2, Azure AD
- **Usar Azure Key Vault o AWS Secrets Manager** para secretos
- **Configurar Application Insights** para monitoreo en producción
- **Realizar auditorías de seguridad** con herramientas .NET
- **Capacitar al equipo** en OWASP Top 10 y secure coding
- **Implementar política de actualizaciones** de paquetes NuGet
- **Configurar health checks** con /health endpoint
- **Documentar la API** con Swagger/OpenAPI incluyendo validaciones
- **Implementar circuit breaker** con Polly para resiliencia
- **Usar User Secrets** en desarrollo, no hardcodear credenciales
- **Habilitar auditoría SQL** en base de datos para compliance
- **Implementar principio de defensa en profundidad** (defense in depth)